路美项目“知识图谱 → 数据查询”两阶段问答架构设计

1. 架构核心思想与知识图谱的价值

在分析了路美项目的前端统计报表页面（`monthly-dashboard` 和 `monthlyReport`）以及深入研究了业界领先的开源 GenBI 项目 WrenAI 后，我们对路美项目的问答架构进行了全面升级。

1.1 知识图谱（语义层）的不可替代性

您提到“用户更多的是问一些统计问题，知识图谱还有用吗？”答案是：**不仅有用，而且是实现高准确率 NL2SQL 的核心基石**。

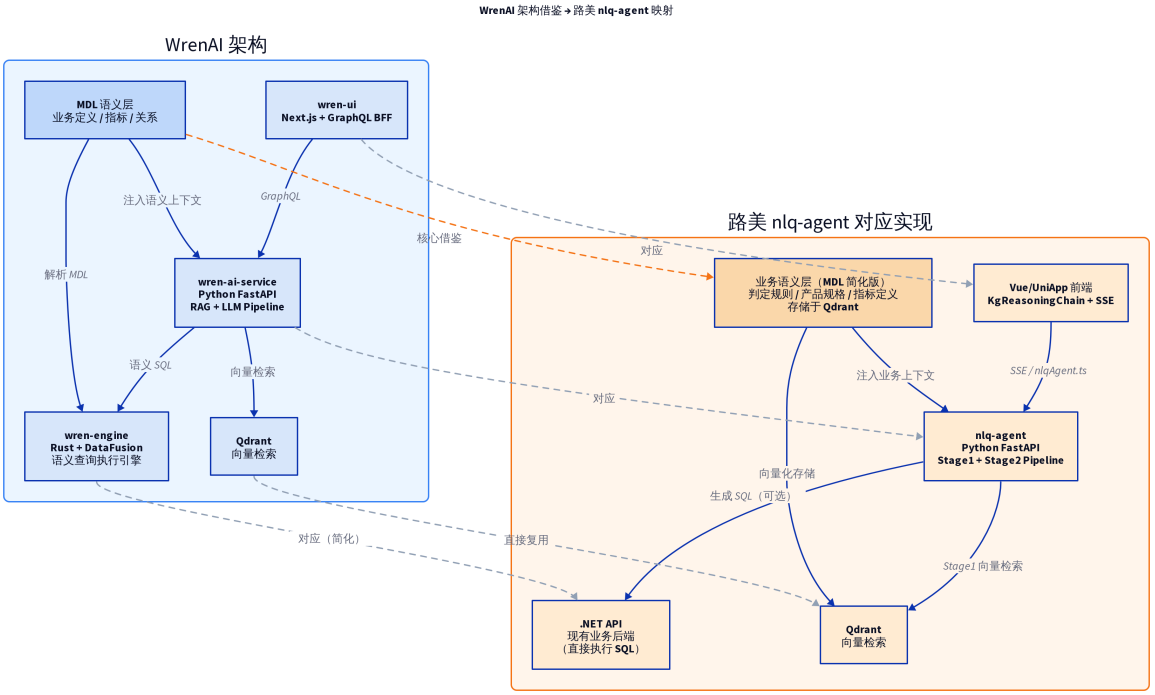
在 WrenAI 的架构中，其最核心的设计理念就是**语义层（Semantic Layer / MDL）** [1]。如果直接将数据库的 DDL（表结构）交给 LLM 生成 SQL，LLM 往往会“猜错”业务逻辑。例如，当用户问“本月 A 类产品合格率是多少？”时，LLM 并不知道：

1. 什么是“A 类产品”？（需要铁损 W15/50 ≤ 某阈值，磁感 B8 ≥ 某阈值）
2. 什么是“合格率”？（是 `qualified_weight / total_weight` 还是其他计算方式？）

因此，**知识图谱在这里扮演的角色正是“业务语义层”**。它将路美项目中的《判定规则》、《产品规格》和《指标定义》结构化地存储在 Qdrant 向量数据库中。在生成 SQL 之前，必须先通过知识图谱检索这些业务定义，这就是我们设计的“先懂业务，再查数据”的两阶段架构。

1.2 借鉴 WrenAI 的架构映射

我们将 WrenAI 的先进理念完美映射到了路美项目的现有技术栈中：

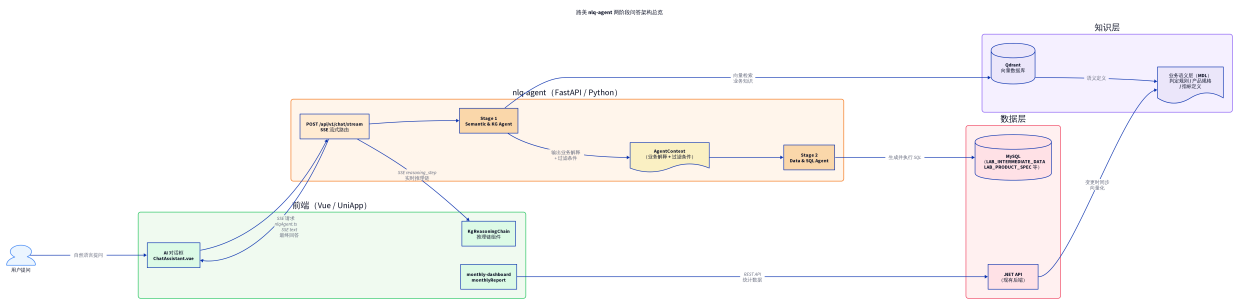


如上图所示，我们利用 Qdrant 构建了轻量级的 MDL 语义层，结合现有的 FastAPI（`nlq-agent`）和 Vue/UniApp 前端组件（`KgReasoningChain`），实现了一个端到端的 GenBI 问答系统。

---

2. 两阶段问答架构总览

我们将用户的提问处理过程明确拆分为两个 Agent（在代码中实现为两个串行的 Pipeline），并通过 SSE 流式协议将中间推理过程实时推送到前端。



2.1 Stage 1: Semantic & KG Agent（语义解析与图谱检索）

- **目标**：理解用户意图，从 Qdrant 向量库中检索出相关的业务规则、名词解释或指标定义。
- **输入**：用户的自然语言问题（如“为什么昨天的合格率这么低？”）。
- **输出**：一段纯文本的业务解释，以及提取出的结构化过滤条件（如阈值、规格要求、指标计算公式）。这些内容将被封装在 `AgentContext` 中传递给下一阶段。
- **前端交互**：在此阶段，Agent 会通过 SSE 下发 `kind: "spec"` 或 `kind: "rule"` 的 `reasoning_step`，前端 `<KgReasoningChain>` 会实时渲染出“正在查询产品规格”、“已找到铁损判定规则”等提示。

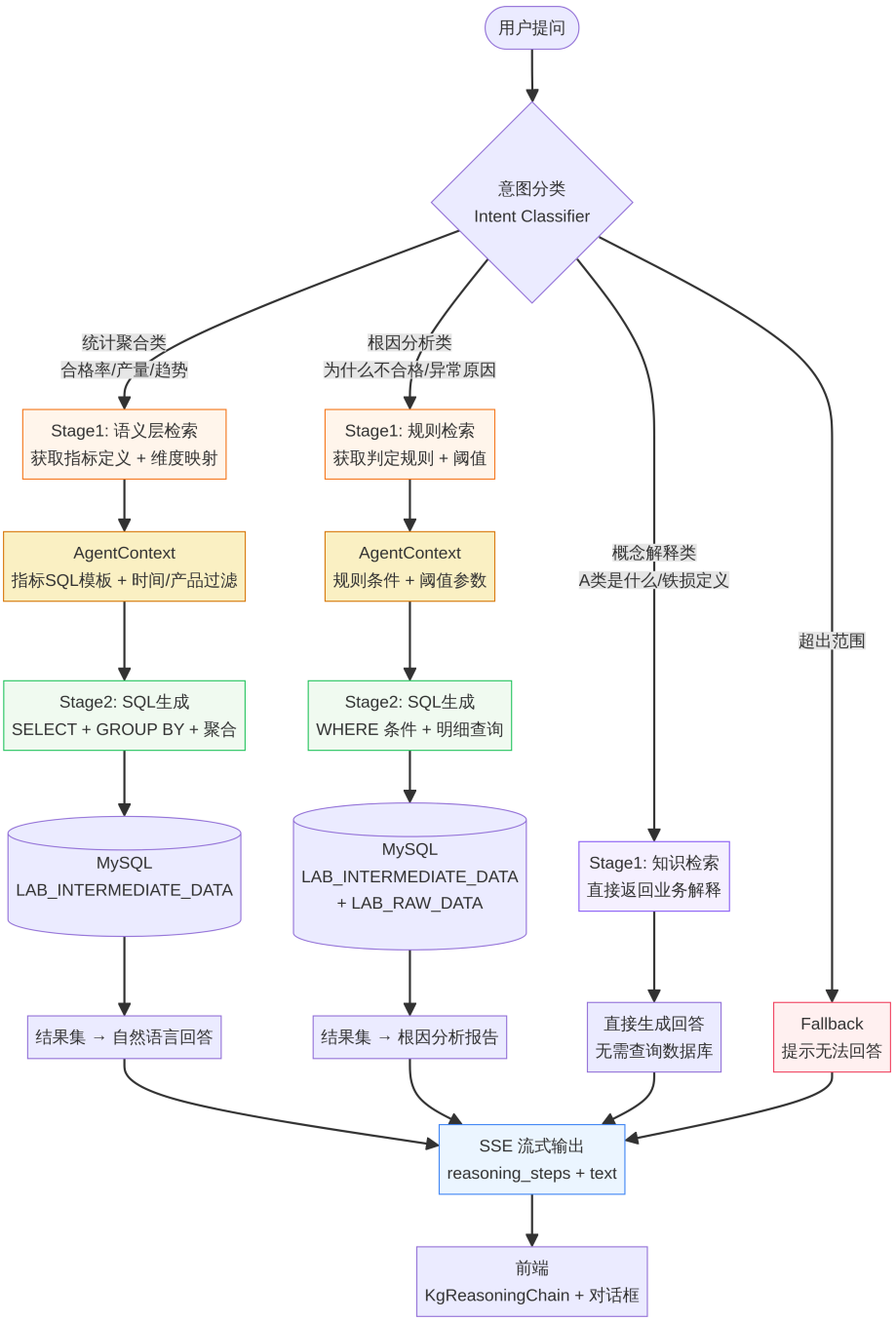
2.2 Stage 2: Data & SQL Agent（数据查询与分析）

- **目标**：基于 Stage 1 提供的“业务解释”和“过滤条件”，结合数据库表结构（DDL），生成 SQL 语句并执行查询。
- **输入**：`AgentContext`（包含业务语义和过滤条件）+ 数据库 DDL。
- **输出**：最终的数据结果，以及结合业务解释生成的自然语言回答。
- **前端交互**：在此阶段，Agent 会通过 SSE 下发 `kind: "condition"`（带 `actual` 和 `satisfied` 字段）以及最终的 `kind: "grade"` 和 `text` 回答。

---

3. 统计问题的智能路由设计

针对前端 `monthly-dashboard` 和 `monthlyReport` 页面中常见的统计类问题，我们设计了智能路由机制。系统会根据用户意图，自动选择不同的处理路径。



3.1 意图分类与处理路径

1. **统计聚合类（如：合格率、产量、趋势）**
   - **Stage 1**：从语义层检索指标的计算公式（如合格率 = `qualified_weight / total_weight`）和维度映射（如班次 = `shift`）。
   - **Stage 2**：生成带有 `GROUP BY` 和聚合函数的 SQL，查询 `LAB_INTERMEDIATE_DATA` 表。
2. **根因分析类（如：为什么不合格、异常原因）**

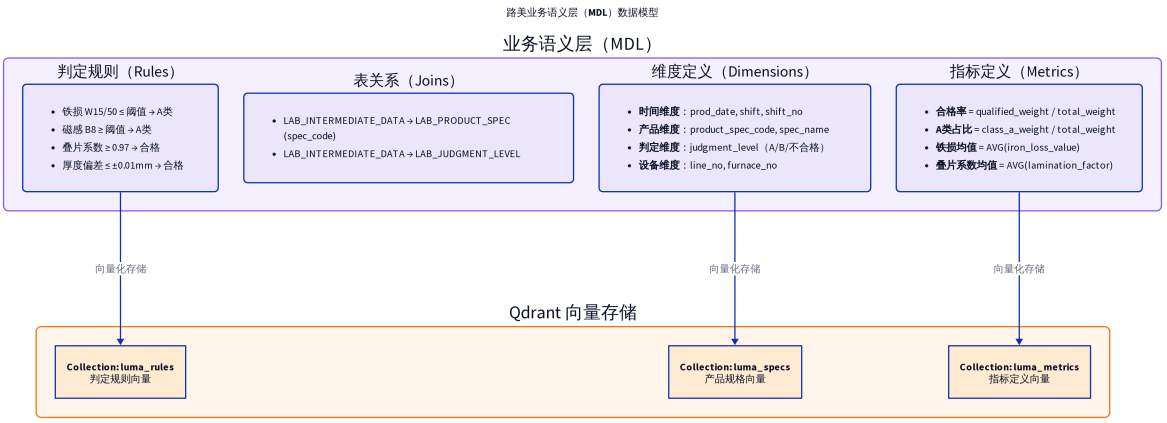
- **Stage 1**：检索具体的判定规则和阈值参数。
- **Stage 2**：生成带有复杂 `WHERE` 条件的 SQL，可能需要关联查询 `LAB_RAW_DATA` 以获取具体的检测值。

3. **概念解释类（如：A 类是什么、铁损的定义）**
    - **Stage 1**：直接从知识图谱获取业务解释。
    - **Stage 2**：跳过 SQL 生成，直接返回文本回答（因为不需要查询动态数据）。

---

4. 语义层（MDL）数据模型设计

为了支撑上述的智能路由和准确的 SQL 生成，我们需要在 Qdrant 中构建以下语义层数据模型：



4.1 核心语义组件

组件类型	描述	示例内容	对应数据库表
指标 (Metrics)	定义业务指标的计算方式	合格率 = qualified_weight / total_weight	无（虚拟计算列）
维度 (Dimensions)	定义数据的分析视角	时间（prod_date）、班次（shift）、设备（line_no）	LAB_INTERMEDIATE_DATA
规则 (Rules)	业务判定的具体阈值条件	铁损 W15/50 ≤ 2.30 → A 类	LAB_JUDGMENT_LEVEL
规格 (Specs)	不同产品的具体要求	牌号 50W470 的厚度标准为 0.5mm	LAB_PRODUCT_SPEC

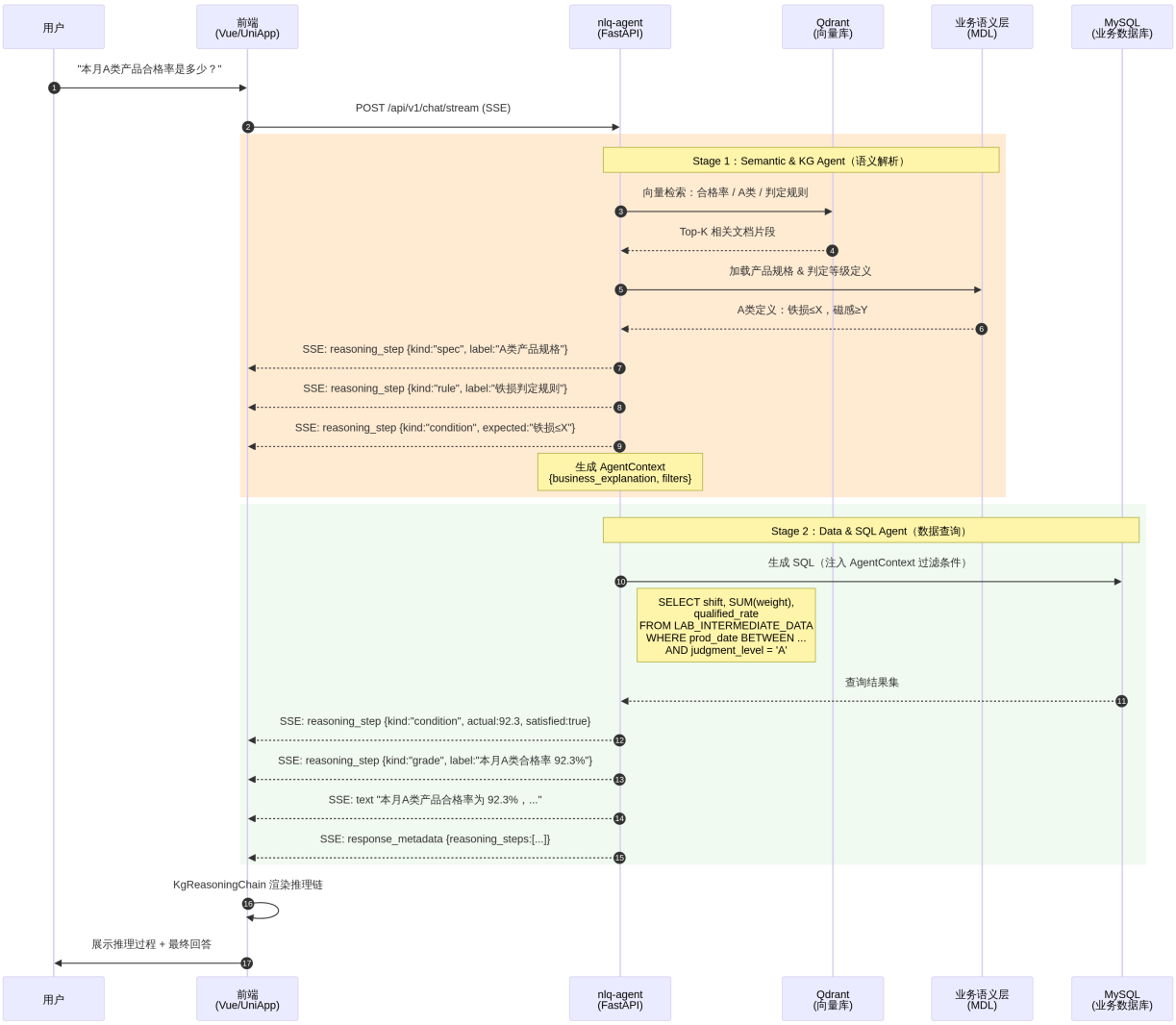
---

5. 前端集成与交互时序

本架构设计的一个重大优势是**完全兼容现有的前端组件**，无需对 `KgReasoningChain` 或 `nlqAgent.ts` 进行任何修改。

5.1 完整交互时序图

以下是用户提问“本月 A 类产品合格率是多少？”时的完整前后端交互时序：


用户
前端
(Vue/UniApp)
nlq-agent
(FastAPI)
Qdrant
(向量库)
业务语义层
(MDL)
MySQL
(业务数据库)
"本月A类产品合格率是多少？"
POST /api/v1/chat/stream (SSE)
Stage 1：Semantic & KG Agent（语义解析）
向量检索：合格率 / A类 / 判定规则
Top-K 相关文档片段
加载产品规格 & 判定等级定义
A类定义：铁损≤X，磁感≥Y
SSE: reasoning_step {kind:"spec", label:"A类产品规格"}
SSE: reasoning_step {kind:"rule", label:"铁损判定规则"}
SSE: reasoning_step {kind:"condition", expected:"铁损≤X"}
生成 AgentContext
{business_explanation, filters}
Stage 2：Data & SQL Agent（数据查询）
生成 SQL（注入 AgentContext 过滤条件）
SELECT shift, SUM(weight),
qualified_rate
FROM LAB_INTERMEDIATE_DATA
WHERE prod_date BETWEEN ...
AND judgment_level = 'A'
查询结果集
SSE: reasoning_step {kind:"condition", actual:92.3, satisfied:true}
SSE: reasoning_step {kind:"grade", label:"本月A类合格率 92.3%"}
SSE: text "本月A类产品合格率为 92.3%，..."
SSE: response_metadata {reasoning_steps:[...]}
KgReasoningChain 渲染推理链
展示推理过程 + 最终回答
用户
前端
(Vue/UniApp)
nlq-agent
(FastAPI)
Qdrant
(向量库)
业务语义层
(MDL)
MySQL
(业务数据库)

5.2 SSE 事件流示例

```
// Stage 1: 语义检索
data: {"type": "reasoning_step", "reasoning_step": {"kind": "spec", "label": "A类产品规格"}}
data: {"type": "reasoning_step", "reasoning_step": {"kind": "rule", "label": "铁损判定规则"}}
data: {"type": "reasoning_step", "reasoning_step": {"kind": "condition", "expected": "铁损≤X"}}

// Stage 2: 数据查询与回填
data: {"type": "reasoning_step", "reasoning_step": {"kind": "condition", "expected": "铁损≤X", "actual": 92.3, "satisfied": true}}
data: {"type": "reasoning_step", "reasoning_step": {"kind": "grade", "label": "本月A类合格率 92.3%"}}

// 最终回答
data: {"type": "text", "content": "本月A类产品合格率为 92.3%，..."}
data: {"type": "response_metadata", "response_payload": {"reasoning_steps": [...]}}
data: {"type": "done"}
```

---

6. 实施路线图

1. **Phase 1: 语义层初始化**
   - 编写 Python 脚本，将现有的 `LAB_PRODUCT_SPEC` 和 `LAB_JUDGMENT_LEVEL` 数据提取并向量化存入 Qdrant。
   - 手动定义常用的统计指标（如合格率、产量）并存入 Qdrant。
2. **Phase 2: Stage 1 Agent 开发**
   - 实现意图分类器。
   - 实现 Qdrant 检索逻辑，生成 `AgentContext`。
3. **Phase 3: Stage 2 Agent 开发**
   - 接入 LLM 进行 NL2SQL 转换。
   - 实现安全的只读 SQL 执行器。

4. **Phase 4: SSE 协议对接**
    - 组装两个 Stage，通过 FastAPI 的 StreamingResponse 输出标准化的 `reasoning_step` 事件。
5. **Phase 5: 数据同步机制**
    - 在 .NET 后端中增加 Hook，当规格或规则发生变更时，通知 `nlq-agent` 更新 Qdrant 向量库。

References

[1] Canner/WrenAI. (2026). Open-source text-to-SQL and text-to-chart GenBI agent with a semantic layer. GitHub. https://github.com/Canner/WrenAI